```
# ============================================================
# LAB: Apriori Algorithm with Varying Support & Confidence
# Datasets: Adult (Census Income) and Epub (E-Book Downloads)
# ============================================================

# ------------------------------------------------------------
# STEP 1: Install and load required packages
# ------------------------------------------------------------
required_pkgs <- c("arules", "arulesViz", "RColorBrewer")
new_pkgs <- required_pkgs[!(required_pkgs %in% installed.packages()[, "Package"])]
if (length(new_pkgs)) install.packages(new_pkgs)

library(arules)     # Core package: apriori(), transactions class
library(arulesViz)  # Visualization of rules
library(RColorBrewer) # Color palettes for plots


# ============================================================
# PART A: Dataset 1 - Adult (Census Income Data)
# ============================================================

# ------------------------------------------------------------
# STEP 2: Load the Adult dataset
# ------------------------------------------------------------
data("Adult")          # Loads built-in transactions object
class(Adult)           # Confirm it's of class "transactions"
Adult                  # Quick summary: number of transactions & items
```

```
summary(Adult)          # Detailed summary: item frequencies, sizes, etc.

# ---------------------------------------------------------
# STEP 3: Inspect the data
# ---------------------------------------------------------
inspect(head(Adult, 3))    # View first 3 transactions in readable form

# Plot the top 15 most frequent items (attributes/categories)
itemFrequencyPlot(Adult, topN = 15, type = "absolute",
            col = brewer.pal(8, "Set2"),
            main = "Top 15 Frequent Items - Adult Dataset",
            cex.names = 0.7)

# ---------------------------------------------------------
# STEP 4: Run Apriori with a baseline support & confidence
# ---------------------------------------------------------
rules1 <- apriori(Adult,
            parameter = list(supp = 0.3,   # minimum support = 30%
                        conf = 0.8,   # minimum confidence = 80%
                        minlen = 2,   # rules need at least 2 items
                        target = "rules"))

summary(rules1)          # Number of rules, distribution of rule lengths
inspect(sort(rules1, by = "lift")[1:5])   # Top 5 rules sorted by lift

# ---------------------------------------------------------
# STEP 5: Vary SUPPORT (confidence fixed at 80%)
# ---------------------------------------------------------
```

```
support_levels <- c(0.5, 0.4, 0.3, 0.2, 0.1)
supp_results <- data.frame(Support = support_levels, Num_Rules = NA)

for (i in seq_along(support_levels)) {
  r <- apriori(Adult,
           parameter = list(supp = support_levels[i],
                            conf = 0.8,
                            minlen = 2,
                            target = "rules"),
           control = list(verbose = FALSE))   # suppress progress output
  supp_results$Num_Rules[i] <- length(r)
}

supp_results   # Table: support level vs. number of rules generated
#>   Support Num_Rules
#> 1    0.5       ..
#> 2    0.4       ..
#> 3    0.3       ..
#> 4    0.2       ..
#> 5    0.1       ..

# Plot: how rule count changes as support decreases
plot(supp_results$Support, supp_results$Num_Rules, type = "b",
    pch = 19, col = "darkblue", lwd = 2,
    xlab = "Minimum Support", ylab = "Number of Rules",
    main = "Effect of Support on Rule Count (Adult, conf = 0.8)")

# ------------------------------------------------------------
```

```
# STEP 6: Vary CONFIDENCE (support fixed at 30%)
# ------------------------------------------------------------
conf_levels <- c(0.95, 0.85, 0.75, 0.6, 0.4)
conf_results <- data.frame(Confidence = conf_levels, Num_Rules = NA)

for (i in seq_along(conf_levels)) {
  r <- apriori(Adult,
           parameter = list(supp = 0.3,
                            conf = conf_levels[i],
                            minlen = 2,
                            target = "rules"),
           control = list(verbose = FALSE))
  conf_results$Num_Rules[i] <- length(r)
}

conf_results   # Table: confidence level vs. number of rules

# Plot: how rule count changes as confidence decreases
plot(conf_results$Confidence, conf_results$Num_Rules, type = "b",
    pch = 19, col = "darkred", lwd = 2,
    xlab = "Minimum Confidence", ylab = "Number of Rules",
    main = "Effect of Confidence on Rule Count (Adult, supp = 0.3)")

# ------------------------------------------------------------
# STEP 7: Grid search over BOTH support and confidence
# ------------------------------------------------------------
grid <- expand.grid(supp = c(0.5, 0.3, 0.2),
                    conf = c(0.9, 0.7, 0.5))
```

```
grid$Num_Rules <- mapply(function(s, c) {
  length(apriori(Adult,
                 parameter = list(supp = s, conf = c, minlen = 2),
                 control = list(verbose = FALSE)))
}, grid$supp, grid$conf)

grid   # Full grid: each support-confidence combo and its rule count

# ------------------------------------------------------------
# STEP 8: Targeted rule mining - what predicts HIGH income?
# ------------------------------------------------------------
rules_income <- apriori(Adult,
                        parameter = list(supp = 0.05, conf = 0.6, minlen = 2),
                        appearance = list(default = "lhs",       # antecedents only
                                          rhs = "income=large"),  # fix the consequent
                        control = list(verbose = FALSE))

rules_income <- sort(rules_income, by = "lift", decreasing = TRUE)
inspect(rules_income[1:10])   # Top 10 predictors of high income

# ------------------------------------------------------------
# STEP 9: Visualize the best rules
# ------------------------------------------------------------
rules_best <- sort(rules1, by = "lift", decreasing = TRUE)

plot(rules_best[1:50])                # scatter: support vs confidence, colored by lift
plot(rules_best[1:10], method = "graph")   # network graph of item relationships
```

```
plot(rules_best[1:10], method = "grouped") # grouped bubble matrix



# ============================================================
# PART B: Dataset 2 - Epub (E-Book Download Transactions)
# ============================================================

# ------------------------------------------------------------
# STEP 10: Load the Epub dataset
# ------------------------------------------------------------
data("Epub")
class(Epub)
Epub            # Sparse dataset: many distinct document IDs
summary(Epub)

# ------------------------------------------------------------
# STEP 11: Inspect the data
# ------------------------------------------------------------
inspect(head(Epub, 5))

itemFrequencyPlot(Epub, topN = 10, type = "relative",
                  col = "skyblue",
                  main = "Top 10 Frequent Documents - Epub Dataset")

# ------------------------------------------------------------
# STEP 12: Run Apriori at multiple support/confidence combos
# (Epub is sparse, so it needs MUCH lower support than Adult)
# ------------------------------------------------------------
```

```
settings <- list(
  c(supp = 0.001,  conf = 0.6),
  c(supp = 0.0005, conf = 0.5),
  c(supp = 0.0002, conf = 0.4)
)

for (s in settings) {
  cat("\n--- Support =", s["supp"], " Confidence =", s["conf"], "---\n")
  r <- apriori(Epub,
               parameter = list(supp = s["supp"], conf = s["conf"], minlen = 2),
               control = list(verbose = FALSE))
  print(summary(r))
}

# ----------------------------------------------------------
# STEP 13: Vary support systematically on Epub
# ----------------------------------------------------------
support_levels <- c(0.002, 0.001, 0.0007, 0.0005, 0.0002)
epub_supp_results <- data.frame(Support = support_levels, Num_Rules = NA)

for (i in seq_along(support_levels)) {
  r <- apriori(Epub,
               parameter = list(supp = support_levels[i], conf = 0.5, minlen = 2),
               control = list(verbose = FALSE))
  epub_supp_results$Num_Rules[i] <- length(r)
}

epub_supp_results
```

```
plot(epub_supp_results$Support, epub_supp_results$Num_Rules, type = "b",
    pch = 19, col = "darkgreen", lwd = 2,
    xlab = "Minimum Support", ylab = "Number of Rules",
    main = "Effect of Support on Rule Count (Epub, conf = 0.5)")

# ------------------------------------------------------------
# STEP 14: Inspect and visualize best Epub rules
# ------------------------------------------------------------
rules_epub <- apriori(Epub,
                      parameter = list(supp = 0.0005, conf = 0.5, minlen = 2),
                      control = list(verbose = FALSE))
rules_epub <- sort(rules_epub, by = "lift", decreasing = TRUE)
inspect(rules_epub[1:10])

if (length(rules_epub) >= 10) {
  plot(rules_epub[1:10], method = "graph", engine = "htmlwidget")
}



# ============================================================
# PART C: Comparative Summary Across Datasets
# ============================================================

# ------------------------------------------------------------
# STEP 15: Build a comparison table
# ------------------------------------------------------------
compare_df <- rbind(
```

```
 data.frame(Dataset = "Adult", Support = 0.3, Confidence = 0.8,
        Rules = length(apriori(Adult, parameter = list(supp = 0.3, conf = 0.8, minlen = 2),
                               control = list(verbose = FALSE)))),
 data.frame(Dataset = "Adult", Support = 0.2, Confidence = 0.5,
        Rules = length(apriori(Adult, parameter = list(supp = 0.2, conf = 0.5, minlen = 2),
                               control = list(verbose = FALSE)))),
 data.frame(Dataset = "Epub", Support = 0.001, Confidence = 0.6,
        Rules = length(apriori(Epub, parameter = list(supp = 0.001, conf = 0.6, minlen = 2),
                               control = list(verbose = FALSE)))),
 data.frame(Dataset = "Epub", Support = 0.0005, Confidence = 0.4,
        Rules = length(apriori(Epub, parameter = list(supp = 0.0005, conf = 0.4, minlen =
2),
                               control = list(verbose = FALSE))))
)

compare_df   # Final side-by-side comparison
```